



Chapter 18

Neural Radiance Field

Lin ZHANG
School of Computer Science and Technology
Tongji University



Outline

- Rendering based on Radiance field
 - Continuous form
 - Discrete form
- Neural Radiance Field (NeRF) and Its Training
- Practice



Rendering based on radiance field

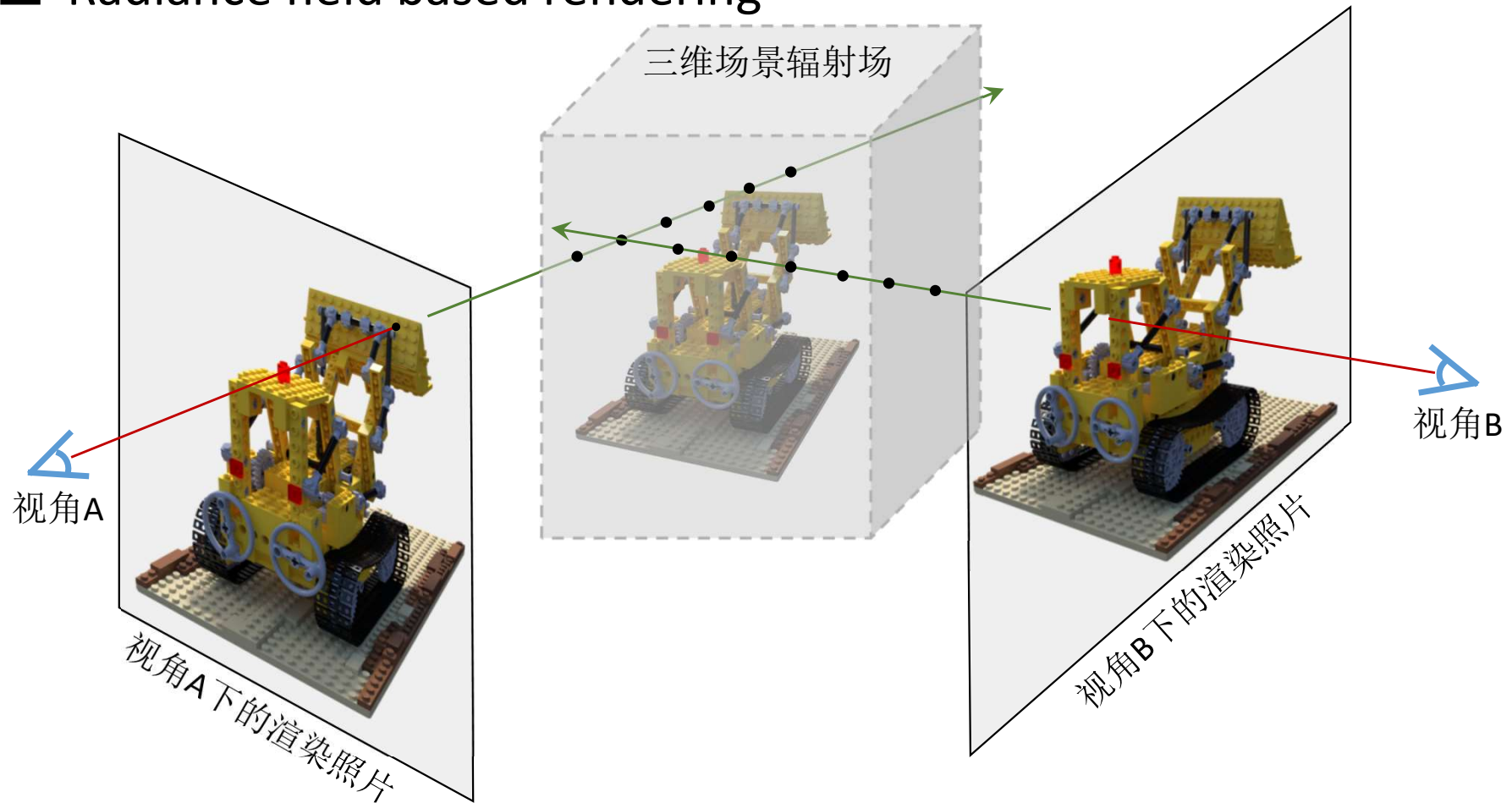
□ Radiance field based rendering

- ✓ This rendering method treats the physical space containing the scene as a radiance field: each point in the space possesses two attributes, **opacity and view-dependent color**
- ✓ The goal of the rendering process is to generate an image of the scene from the **perspective of a virtual camera**
- ✓ The viewpoint of the virtual camera is defined by its pose in the world coordinate system. The essence of the rendering operation is to **determine the color of each point on the imaging plane of the virtual camera**



Rendering based on radiance field

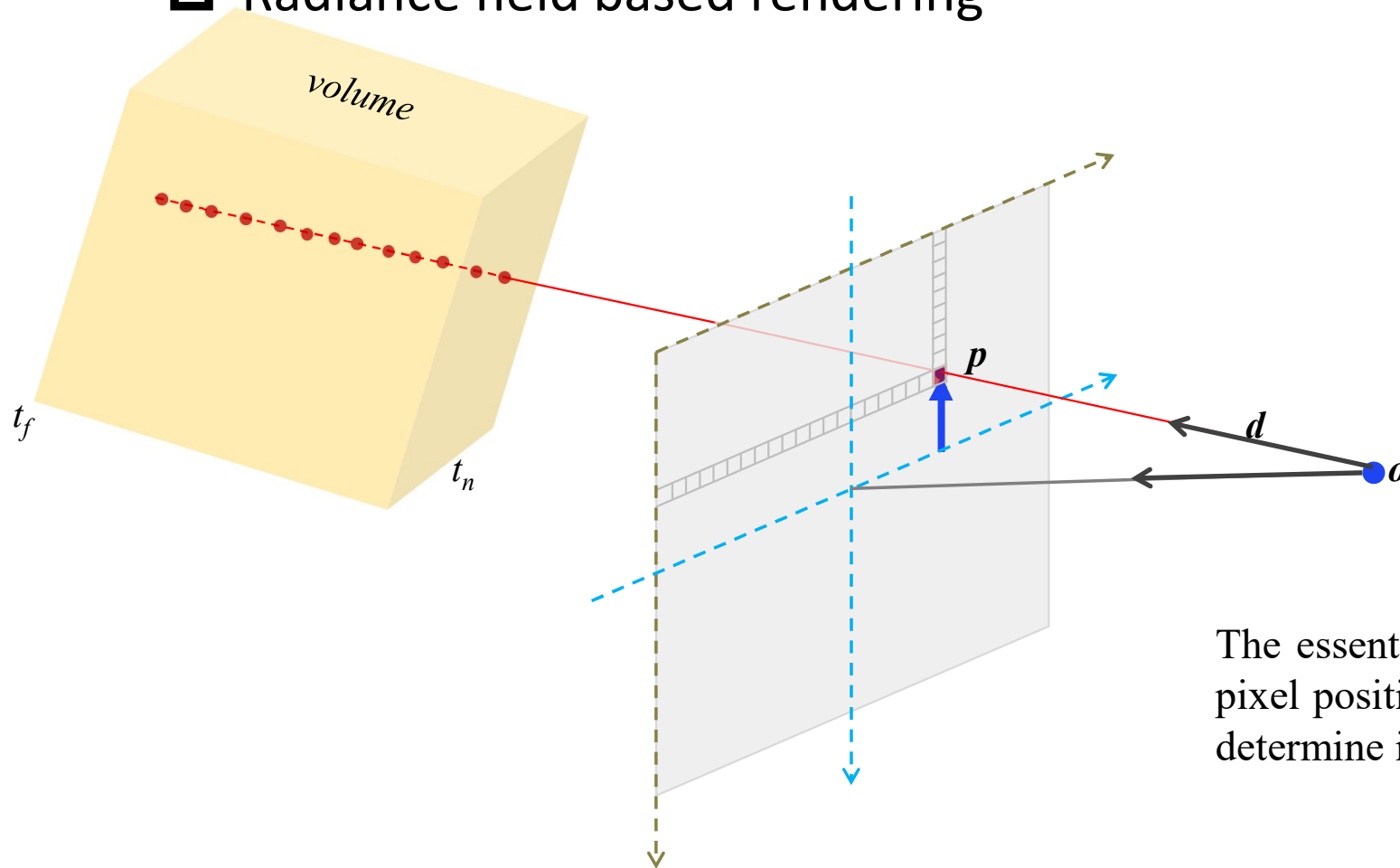
□ Radiance field based rendering





Rendering based on radiance field

□ Radiance field based rendering



The essential question is: for a given pixel position p on the image, how to determine its color?



Outline

- Rendering based on Radiance field
 - Continuous form
 - Discrete form
- Neural Radiance Field (NeRF) and Its Training
- Practice



Rendering based on radiance field—Continuous form

p 's color can be calculated as,

$$\hat{\mathbf{u}}(\mathbf{p}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt \quad (1)$$

where $\sigma(\mathbf{x})$ represents the opacity (不透明度) at the spatial location $\mathbf{x}$

$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ ($t \in [t_n, t_f]$) is one point along the ray $\overrightarrow{\mathbf{op}}$, parameterized by t ; $\mathbf{o}$ is the camera center; $\mathbf{d}$ is the unit vector of the ray $\overrightarrow{\mathbf{op}}$

$\mathbf{c}(\mathbf{r}(t), \mathbf{d})$ is the color of the spatial position $\mathbf{r}(t)$ when viewed along the direction $\mathbf{d}$

$T(t)$ is the accumulated transparency along the ray $\overrightarrow{\mathbf{op}}$ from the point $\mathbf{r}(t_n)$ to $\mathbf{r}(t)$,

$$T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$$



Rendering based on radiance field—Continuous form

With Eq. (1), to compute the color $\hat{\mathbf{u}}(\mathbf{p})$, we need to compute an integral along the ray $\overrightarrow{op}$, which is not easy to be implemented

Need to find a way to convert
the integral to summation



The discrete form of the radiance field based rendering



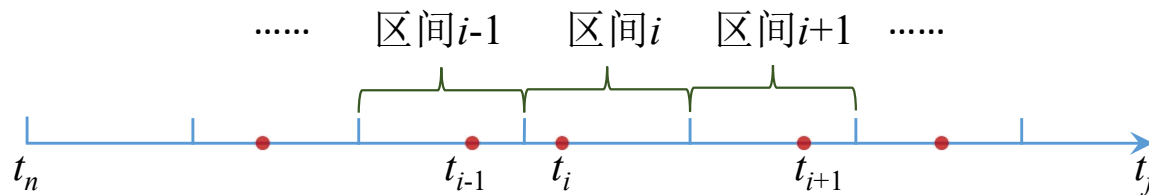
Outline

- Rendering based on Radiance field
 - Continuous form
 - Discrete form
- Neural Radiance Field (NeRF) and Its Training
- Practice



Rendering based on radiance field—Discrete form

Divide the range $[t_n, t_f]$ to N sub-intervals,



The i th sub-interval is $\left[t_n + \frac{i-1}{N}(t_f - t_n), t_n + \frac{i}{N}(t_f - t_n) \right], i = 1, \dots, N$

In sub-interval i , randomly sample one point t_i whose value conforms to a uniform distribution,

$$t_i \sim U \left[t_n + \frac{i-1}{N}(t_f - t_n), t_n + \frac{i}{N}(t_f - t_n) \right]$$

Then, Eq. (1) can be approximated as,

$$\hat{\mathbf{u}}(\mathbf{p}) \approx \sum_{i=1}^{N-1} \hat{\mathbf{u}}_i(\mathbf{p}) = \sum_{i=1}^{N-1} \int_{t_i}^{t_{i+1}} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt \quad (2)$$

where $\hat{\mathbf{u}}_i(\mathbf{p})$ expresses the contribution of the “scenery” encountered by the light between $\mathbf{r}(t_i)$ and $\mathbf{r}(t_{i+1})$ to the final color of point $\mathbf{p}$



Rendering based on radiance field—Discrete form

By considering the opacity and the color in the sub-interval $[t_i, t_{i+1}]$ as constants,

$$\sigma_i = \sigma(\mathbf{r}(t_i)) \quad \mathbf{c}_i = \mathbf{c}(\mathbf{r}(t_i), \mathbf{d})$$

We can have,

$$\hat{\mathbf{u}}_i(\mathbf{p}) = \int_{t_i}^{t_{i+1}} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt \approx \int_{t_i}^{t_{i+1}} \exp\left(-\int_{t_n}^t \sigma(s) ds\right) \sigma_i \mathbf{c}_i dt \quad (3)$$

$$\hat{\mathbf{u}}_i(\mathbf{p}) \approx \sigma_i \mathbf{c}_i \int_{t_i}^{t_{i+1}} \exp\left(-\int_{t_n}^t \sigma(s) ds\right) dt = \sigma_i \mathbf{c}_i \int_{t_i}^{t_{i+1}} \exp\left(-\int_{t_n}^{t_i} \sigma(s) ds\right) \exp\left(-\int_{t_i}^t \sigma(s) ds\right) dt = \sigma_i \mathbf{c}_i T_i \int_{t_i}^{t_{i+1}} \exp\left(-\int_{t_i}^t \sigma(s) ds\right) dt \quad (4)$$

$T_i \triangleq \exp\left(-\int_{t_n}^{t_i} \sigma(s) ds\right)$ will be discretized in next page

$$\int_{t_i}^{t_{i+1}} \exp\left(-\int_{t_i}^t \sigma(s) ds\right) dt \approx \int_{t_i}^{t_{i+1}} \exp(-\sigma_i(t-t_i)) dt = \frac{\exp(-\sigma_i(t-t_i))}{-\sigma_i} \Big|_{t_i}^{t_{i+1}} = \frac{1}{\sigma_i} [1 - \exp(-\sigma_i(t_{i+1}-t_i))]$$

$$\hat{\mathbf{u}}_i(\mathbf{p}) \approx \sigma_i \mathbf{c}_i T_i \int_{t_i}^{t_{i+1}} \exp\left(-\int_{t_i}^t \sigma(s) ds\right) dt \approx \sigma_i \mathbf{c}_i T_i \frac{1}{\sigma_i} [1 - \exp(-\sigma_i(t_{i+1}-t_i))] = \mathbf{c}_i T_i [1 - \exp(-\sigma_i \delta_i)] \quad (5)$$



Rendering based on radiance field—Discrete form

Based on Eq. (2) and Eq. (5), we have,

$$\hat{u}(p) \approx \sum_{i=1}^{N-1} \hat{u}_i(p) = \sum_{i=1}^{N-1} c_i T_i [1 - \exp(-\sigma_i \delta_i)] \quad (6)$$

Discretize the computation of T_i ,

$$\begin{aligned} T_i &= \exp\left(-\int_{t_n}^{t_i} \sigma(s) ds\right) \approx \exp\left(-\left[\sigma_1(t_2 - t_1) + \sigma_2(t_3 - t_2) + \dots + \sigma_{i-1}(t_i - t_{i-1})\right]\right) \\ &= \exp\left(-\sum_{j=1}^{i-1} \sigma_j(t_{j+1} - t_j)\right) = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right) \end{aligned} \quad (7)$$

Eq. (6) and Eq. (7) together give the discrete form of the continuous rendering model described by Eq. (1)



Outline

- Rendering based on Radiance field
 - Continuous form
 - Discrete form
- Neural Radiance Field (NeRF) and Its Training
- Practice



NeRF and its training

Radiance field can be naturally represented as a mapping function,

$$F_{\Theta}(\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$$

where F_{Θ} is the mapping function, Θ is the parameter set of F ; $\mathbf{x}$ is a 3D point in the radiance field while $\mathbf{d}$ is the viewing direction; σ is the opacity of $\mathbf{x}$ in the radiance field, and $\mathbf{c}=(r,g,b)$ is the color of $\mathbf{x}$ when viewed along the direction of $\mathbf{d}$

For a scene to be rendered, if its associated mapping model F_{Θ} is fixed, its radiance field is fixed or in other words, its radiance field is implicitly represented by F_{Θ}

F_{Θ} can be naturally represented as a neural network; **when a radiance field is represented by a neural network, it is called a Neural Radiance Field (NeRF)**^[1]

[1] MILDENHALL B, SRINIVASAN P P, TANCIK M, et al. NeRF: Representing scenes as neural radiance fields for view synthesis[C]//Proc. European Conf. Computer Vision, 2020: 405–421.



NeRF and its training

Theoretically, F_{θ} maps a $6d$ input vector $(\mathbf{x}, \mathbf{d})$ to a $4d$ vector $(\mathbf{c}, \sigma)$. However, such a straightforward idea cannot characterize **high-frequency information** well



Dimensionality-increasing encoding for the position $\mathbf{x}$ and the direction $\mathbf{d}$

Such a process can be represented as $\gamma(p): \mathbb{R} \rightarrow \mathbb{R}^{2L}$:

$$\gamma(p) = (\sin(2^0 \pi p), \cos(2^0 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p))$$

In the reference implementation:

For position encoding, $L=10$. Thus, after such a dimensionality-increasing encoding process, the data dimension for the positional data is $2 \times 10 \times 3 + 3 = 63$

For direction encoding, $L=4$. Thus, after such a dimensionality-increasing encoding process, the data dimension related to direction is $2 \times 4 \times 3 + 3 = 27$



NeRF and its training

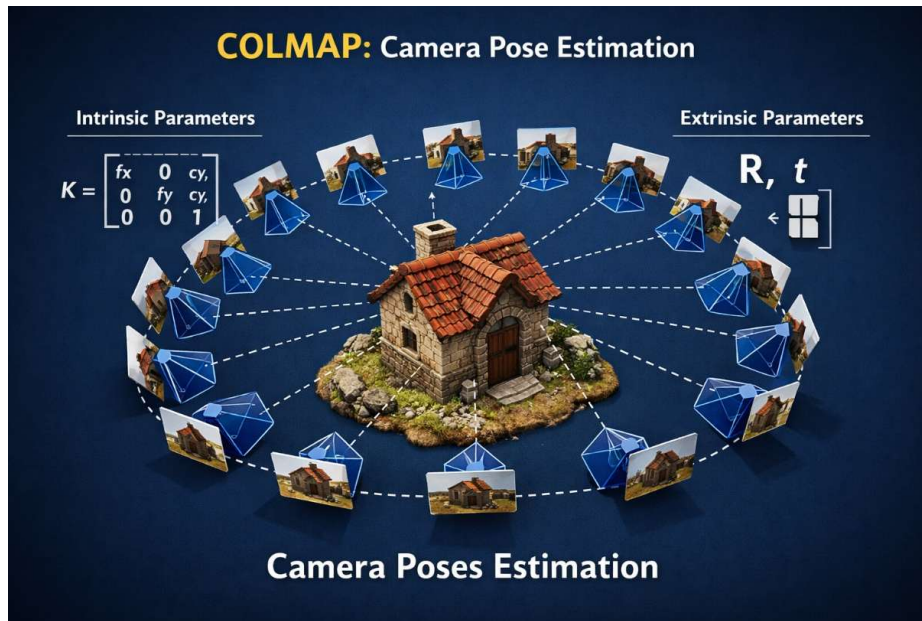


The structure of the neural network F_{θ} for implicitly representing the radiance field



NeRF and its training

- The training strategy of NeRF, i.e., to determine the parameter set θ of the neural network F_{θ}
 - Collect a set of images $\mathcal{P}$ for the scene to be rendered; for each image in $\mathcal{P}$, the associated intrinsics and extrinsics of the camera capturing it are known (usually estimated using some SfM software, e.g. COLMAP)



<https://github.com/colmap/colmap>



NeRF and its training

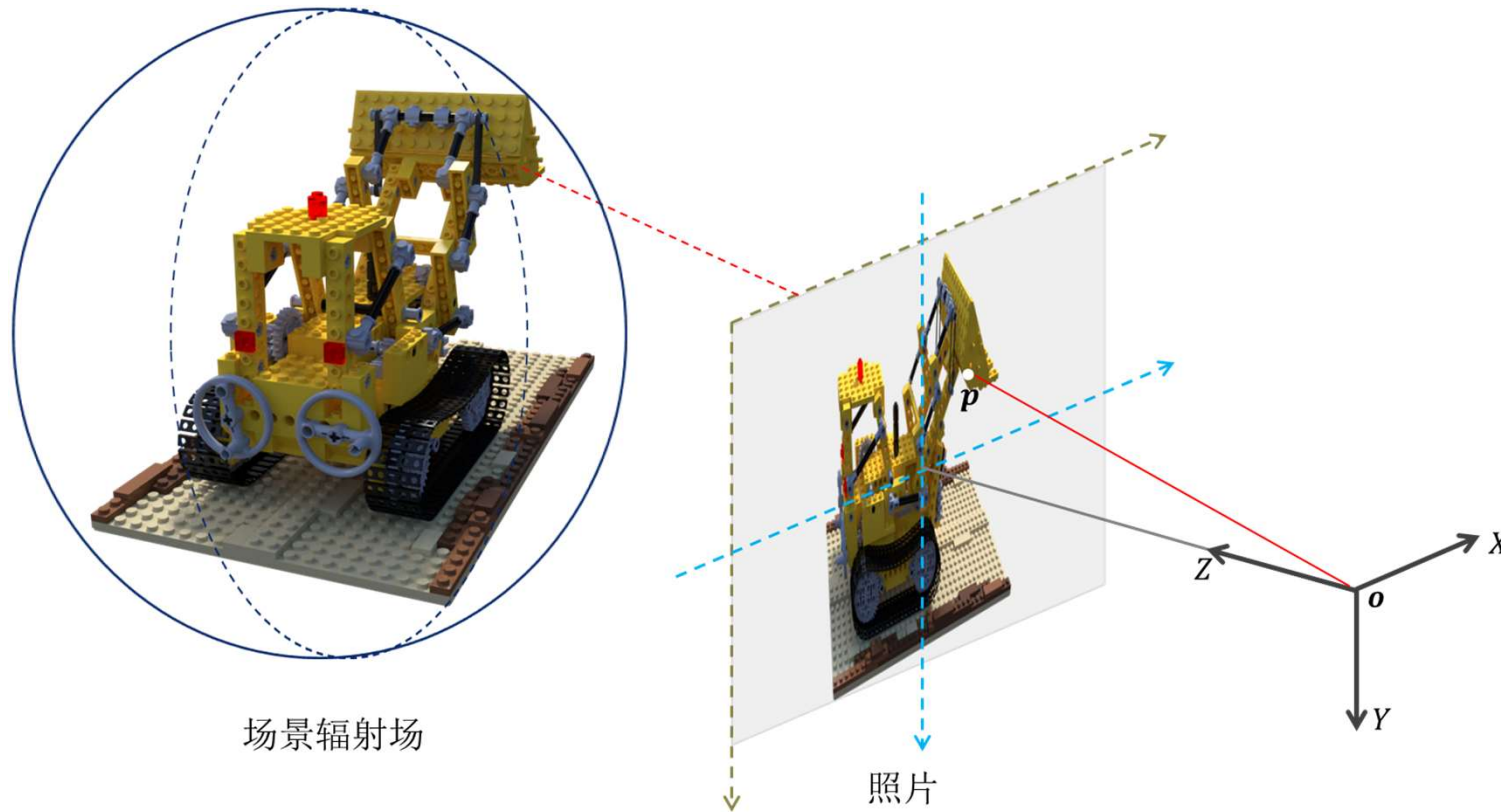
- The training strategy of NeRF, i.e., to determine the parameter set Θ of the neural network F_{Θ}
 - Collect a set of images $\mathcal{P}$ for the scene to be rendered; for each image in $\mathcal{P}$, the associated intrinsics and extrinsics of the camera capturing it are known (usually estimated using some SfM software, e.g. COLMAP)
 - The training loss for F_{Θ} is designed as,

$$l(\Theta) = \sum_{p \in \Omega} \left\| \hat{\mathbf{u}}(\mathbf{p}) - \mathbf{u}(\mathbf{p}) \right\|_2^2$$

where Ω is the pixel set from the image set $\mathcal{P}$; $\mathbf{u}(\mathbf{p})$ is the ground-truth color of pixel $\mathbf{p}$; $\hat{\mathbf{u}}(\mathbf{p})$ is the estimated color for the pixel $\mathbf{p}$ with the radiance field F_{Θ}



NeRF and its training



For one pixel p on training image I , its ground-truth color on I is known as $u(p)$. According to the rendering equation Eq. (1), taking F_{Θ} as the implicit radiance field, p 's color can be estimated as $\hat{u}(p)$. Thus, the loss term can be designed as $\|\hat{u}(p) - u(p)\|_2^2$



NeRF and its training

- The training strategy of NeRF, i.e., to determine the parameter set Θ of the neural network F_{Θ}
 - Collect a set of images $\mathcal{P}$ for the scene to be rendered; for each image in $\mathcal{P}$, the associated intrinsics and extrinsics of the camera capturing it are known (usually estimated using some SfM software, e.g. COLMAP)
 - The training loss for F_{Θ} is designed as,

$$l(\Theta) = \sum_{p \in \Omega} \left\| \hat{\mathbf{u}}(\mathbf{p}) - \mathbf{u}(\mathbf{p}) \right\|_2^2$$

where Ω is the pixel set from the image set $\mathcal{P}$; $\mathbf{u}(\mathbf{p})$ is the ground-truth color of pixel $\mathbf{p}$; $\hat{\mathbf{u}}(\mathbf{p})$ is the estimated color for the pixel $\mathbf{p}$ with the radiance field F_{Θ}

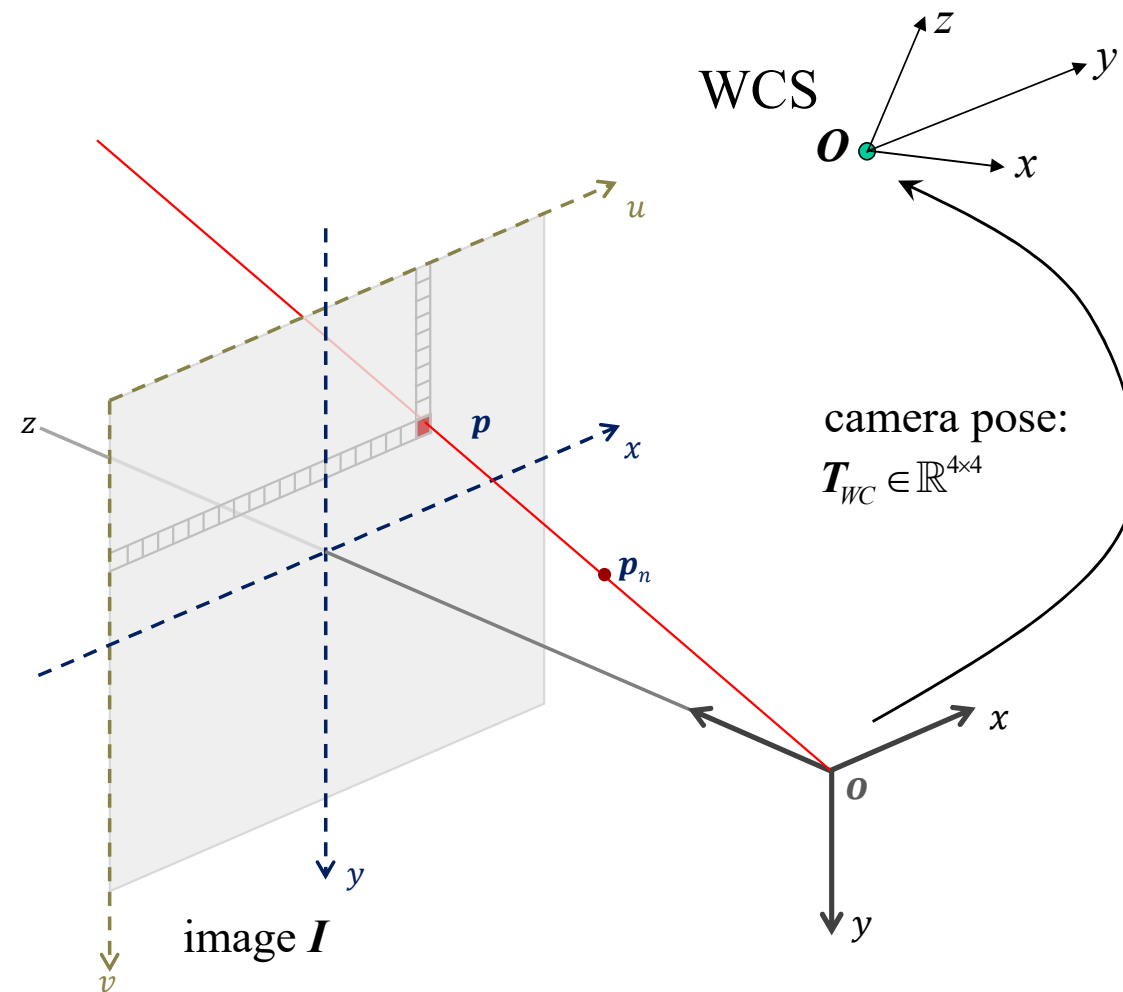
How? 

$$\hat{\mathbf{u}}(\mathbf{p}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt \quad (1)$$

The key issue is to determine the ray $\overrightarrow{\mathbf{o}\mathbf{p}}$, pointing from the camera center $\mathbf{o}$ to pixel $\mathbf{p}$



NeRF and its training



p (homogeneous form) is a pixel on image I , whose camera's pose w.r.t. the WCS is $T_{WC} \in \mathbb{R}^{4 \times 4}$
 p 's corresponding point p_n on the normalized retinal plane is $K^{-1}p$, where K is the intrinsics matrix of the camera; p_n can also be regarded as a 3D point w.r.t. the camera coordinate system

As a 3D point, p_n 's coordinate w.r.t. the WCS is,

$$p_w \triangleq T_{WC} p_n$$

o 's coordinate w.r.t. the camera coordinate system is $(0,0,0,1)^T$ and its coordinate w.r.t. the WCS is,

$$o_w \triangleq T_{WC} (0,0,0,1)^T$$

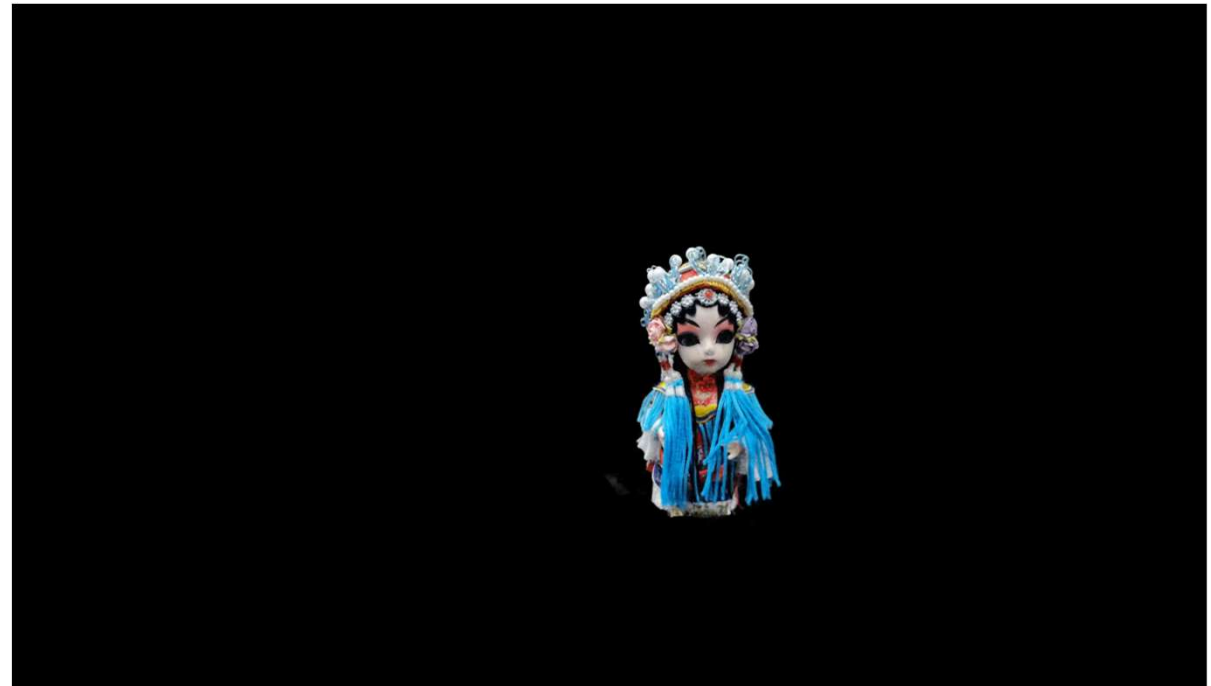


Then, we have the ray $\overrightarrow{op}$'s representation w.r.t. the WCS, $\overrightarrow{o_w p_w}$



NeRF rendering results

- When the NeRF model of a scene is trained, any view of this scene can be rendered





Outline

- Rendering based on Radiance field
 - Continuous form
 - Discrete form
- Neural Radiance Field (NeRF) and Its Training
- Practice



Instant-NGP

- NeRF is not so efficient and thus it is quite slow to train
- Instant-NGP^[2] can be seen as an improved version for NeRF; it runs much faster than the original NeRF
- So in practice section, we choose Instant-NGP (whose details may be covered in the 2nd edition of this textbook☺)
- The project homepage for instant-ngp is <https://github.com/NVlabs/instant-ngp/>

[2] MÜLLER T, EVANS A, SCHIED C, et al. Instant neural graphics primitives with a multiresolution hash encoding[J]. ACM Trans. Graphics, 2022, 41(4): 102:1-15.



Instant-NGP

• 编译Instant-NGP项目

- 作者所用的基础开发环境为：Windows 11操作系统，NVIDIA GeForce RTX 4090 GPU计算卡，CUDA 12.8+cuDNN，Visual Studio 2022，CMake 4.1.0-rc1，Git-2.50.0-64-bit

以管理员身份启动 cmd 命令行窗口，并定位到要存储 instant-ngp 项目的路径。然后在 cmd 中执行，↵

```
git clone --recursive https://github.com/nvlab/instant-ngp↵
```

该命令会下载 instant-ngp 项目源代码到本地。然后在 cmd 窗口中执行，↵

```
cd instant-ngp↵
```

```
cmake . -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo↵
```

```
cmake --build build --config RelWithDebInfo -j↵
```

如上的 cmake 命令会执行编译过程，调用 Visual Studio 编译器完成对 Instant-NGP 项目的编译。编译过程顺利完成后，会在\instant-ngp\目录下出现可执行文件 instant-ngp.exe。↵



Instant-NGP

• 创建Instant-NGP项目所需Python运行环境

为了要顺利运行 instant-ngp 项目提供的一些 python 处理脚本，需要为其准备 Python 运行环境。↵

以管理员身份启动 Anaconda Prompt。在 Anaconda 中创建并激活虚拟环境“instant-ngp”：↵

```
conda create -n instant-ngp python=3.9↵
```

```
conda activate instant-ngp↵
```

接下来，需要安装运行 instant-ngp 所需要的依赖包。在 Anaconda Prompt 中执行，↵

```
cd path-to-ngp↵
```

```
pip install -r requirements.txt↵
```

接下来需要下载 COLMAP 和 FFMPEG 库。在 Anaconda Prompt 中执行，↵

```
cd scripts↵
```

```
download_colmap.bat1↵
```

```
download_ffmpeg.bat↵
```

执行完毕之后，会在“<path-to-ngp>\external\”目录下出现 colmap 和 ffmpeg 两个文件夹，分别存放了下载的 COLMAP 程序和 FFMPEG 程序。↵

^[1]默认情况下，download_colmap.bat脚本会下载CPU版本的COLMAP程序。如果读者的主机上安装有GPU计算卡并正确安装了CUDA库，可以将download_colmap.bat脚本中出现的“COLMAP-3.7-windows-no-cuda.zip”修改为“COLMAP-3.7-windows-cuda.zip”，来使得该脚本下载GPU版本的COLMAP程序。



Instant-NGP

- 数据的采集与预处理

- 输入数据可以是一段视频，也可以是拍摄的图像集合。需要注意的是，在视频或图像的拍摄过程中要保持相机的焦距不变，也就是要使用定焦模式来拍摄
- 如果采集的数据是视频：
 - » 将模型视频命名为test.mp4，并放置在目录<path-to-ngp>\data\mydata\之下
 - » 在Anaconda Prompt中激活instant-ngp环境并定位到工作目录<path-to-ngp>，然后执行，

```
python scripts\colmap2nerf.py --video_in data\mydata\test.mp4 --video_fps 2 --run_colmap --aabb_scale 16 --overwrite
```

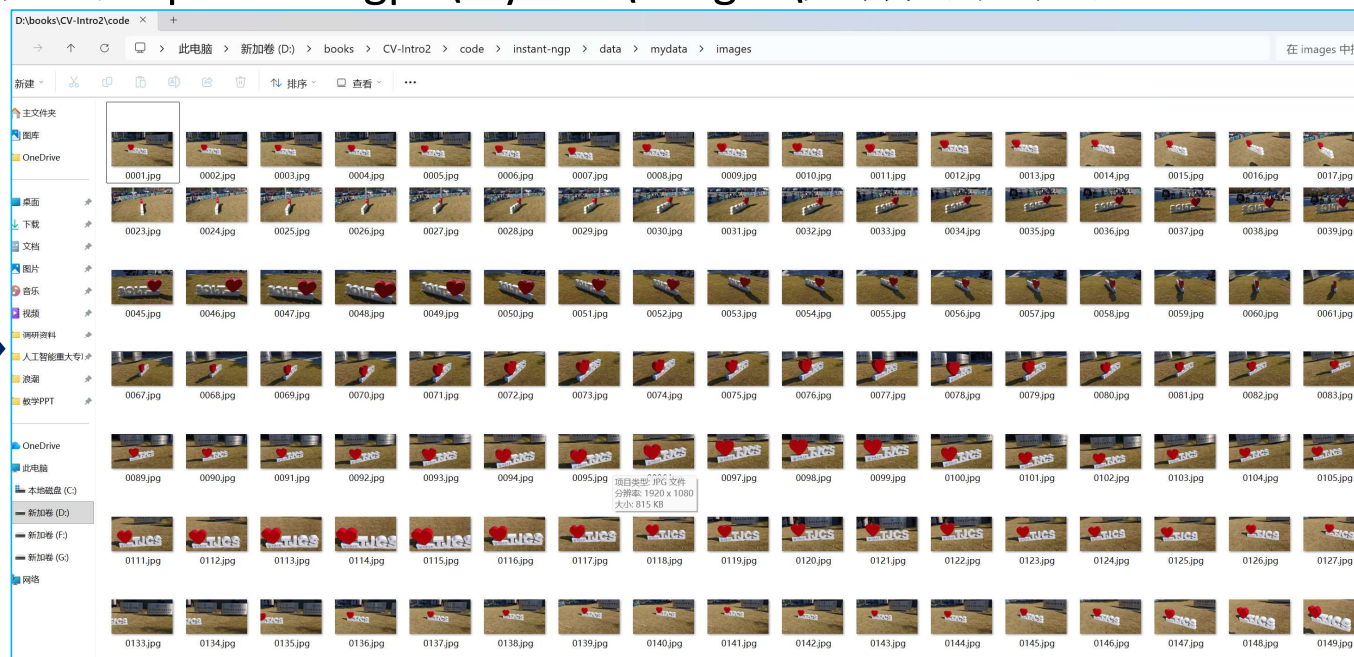
上述指令会以每秒2帧的采样间隔从视频文件<path-to-ngp>\data\mydata\test.mp4中提取出图像帧集合，并将提取出的图像帧集合存储在\data\mydata\images\文件夹中



Instant-NGP

• 数据的采集与预处理

- 输入数据可以是一段视频，也可以是拍摄的图像集合。需要注意的是，在视频或图像的拍摄过程中要保持相机的焦距不变，也就是要使用定焦模式来拍摄
- 如果采集的数据是图像集合：
 - » 直接将所拍摄的图像放置在<path-to-ngp>\mydata\images\文件夹下即可



拍摄的原始视频



Instant-NGP

- 数据的采集与预处理

- 图像数据准备好后，用 COLMAP 软件计算出对应于每个视频帧的相机位姿。在 Anaconda Prompt 中执行，

```
cd <path-to-ngp>data\mydata
```

```
python ../../scripts/colmap2nerf.py --colmap_matcher exhaustive --run_colmap --aabb_scale 16 --overwrite
```

成功执行上述命令后，会在 “<path-to-ngp>\data\mydata” 目录之下生成一个名为 “transforms.json” 的文件，它记录了相机的内参数以及每个采样帧所对应的相机外参



Instant-NGP

- Instant-NGP模型的训练与结果导出

- 已经完成了训练Instant-NGP模型的数据准备。在Anaconda Prompt中执行，

- ```
cd <path-to-ngp>
```

- ```
instant-ngp.exe \data\mydata
```

启动instant-ngp图形界面。点击“start training”即可开始神经辐射场的训练





Instant-NGP

- Instant-NGP模型的训练与结果导出

当用户对当前训练结果满意后，可以点击“stop training”来停止训练过程。此时，可以点击 snapshot 面板处的“save”按钮，来将当前模型权重保存为一个格式为.ingp 的文件。默认情况下，该文件名为“base.ingp”，被保存在了“<path-to-ngp>\data\mydata\”目录之下。我们可以启动 instant-ngp，之后可导入事先保存后的.ingp 模型文件，在 Anaconda Prompt 中执行，↵

```
cd path-to-ngp↵
```

```
instant-ngp.exe↵
```

然后，在 instant-ngp 程序 GUI 界面中 Snapshot 文件名框中输入“\data\mydata\base.ingp”，然后点击 load，就可以导入“<path-to-ngp>\data\mydata\base.ingp”模型权重文件，进行模型的实时渲染。也可以在导入场景的同时导入已经保存的模型权重文件，这样可以支持在已保存的模型权重基础之上继续进行训练，相应的命令行为，↵

```
instant-ngp.exe --scene data\mydata\transforms.json --snapshot=data\mydata\base.ingp↵
```



Instant-NGP

- Instant-NGP模型的训练与结果导出

- “world transform & crop box” 工具可以用来调整模型观察包围盒的大小。通过调整包围盒，可以去除掉不想要的渲染区域，保留比较干净整洁的渲染模型

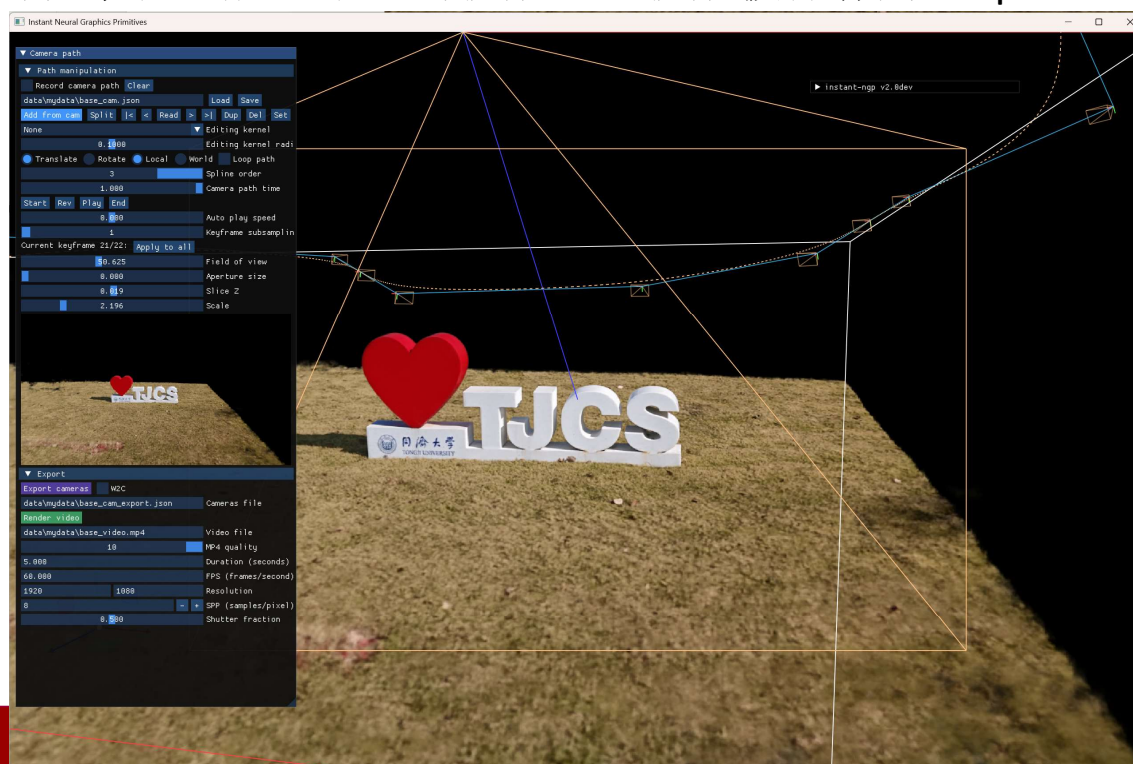




Instant-NGP

• Instant-NGP模型的训练与结果导出

- Instant-NGP提供渲染视频导出功能。通过“Camera path”面板设置和增加关键帧的观察相机位姿，点击“Add from cam”增加一个关键帧观察位置。渲染视频参数设置，视频时长（Duration）、帧率（FPS）和分辨率（resolution）。点击“Render video”，生成沿着关键帧相机位置插值出的观察轨迹所渲染出的视频，该视频被保存为“<path-to-ngp>\video.mp4”。





Instant-NGP

- Instant-NGP模型的训练与结果导出



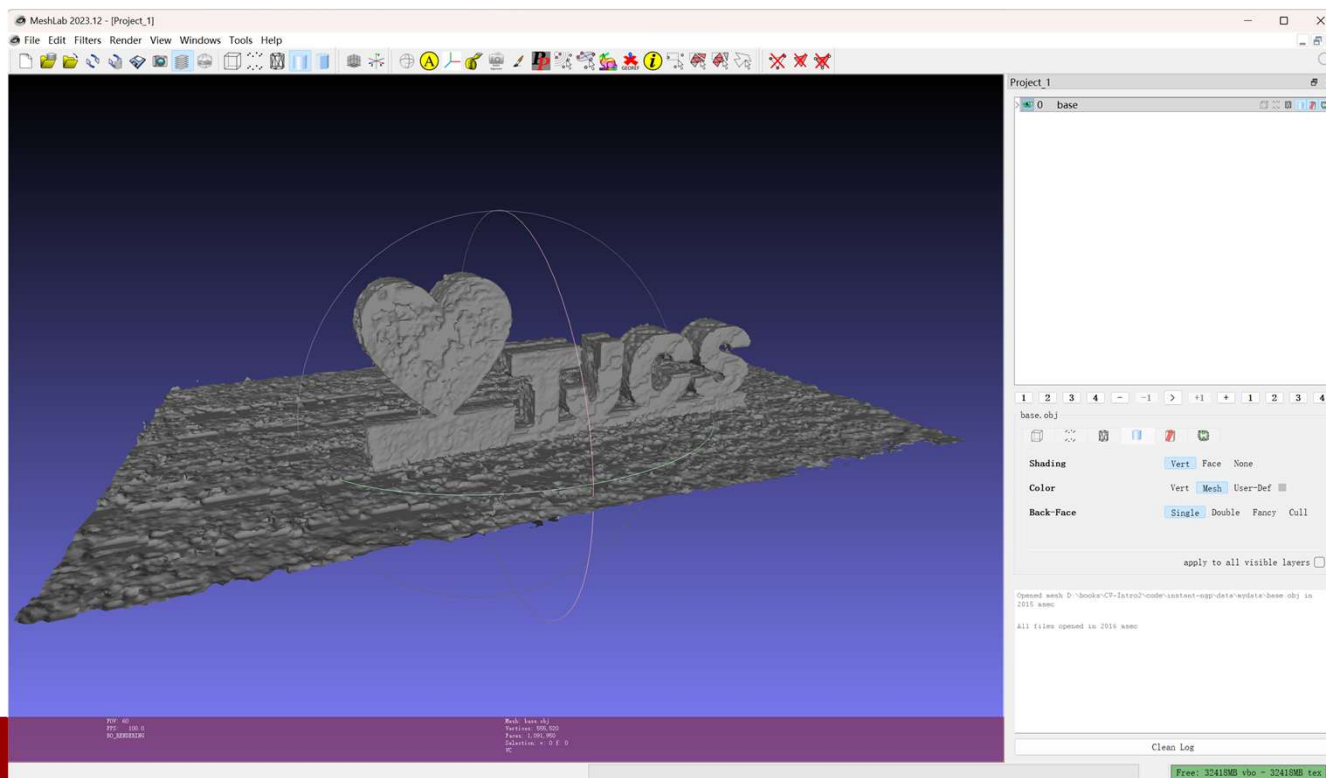
Instant-NGP指定相机观察路径后，渲染视频导出示例



Instant-NGP

• Instant-NGP模型的训练与结果导出

- Instant-NGP也提供神经辐射场面片模型的导出功能，通过面板“Export mesh/volumes/slices”实现；其中，参数“Res”用来控制体素的分辨率，“MC density threshold”是用来判断某体素是否被占据的阈值。点击“Mesh it!”可以完成模型神经辐射场的面片化，点击“save it!”，可以将面片模型保存为文件“<path-to-ngp>\data\mydata\base.obj”。可以用meshlab来打开保存的面片文件base.obj，



在meshlab中打开Instant-NGP导出的面片表示模型

